

EDADSL Documentation

Andreas G.

15th of April 2026

Contents

1	Introduction	7
1.1	Abstract	7
1.2	Reading this Document	7
I	Core Language	8
2	Design Principles	9
2.1	Design Goals	9
2.2	Applications and Application-Areas with special Priority during Design . . .	9
2.3	Design Philosophy	10
3	Lexical Structure	11
3.1	Character Set	11
3.2	Whitespace	11
3.3	Comments	11
3.4	Identifiers	11
3.5	Keywords	11
3.6	Literals	12
3.7	Operators	12
3.8	Delimiters	12
3.9	Tokenization Rules	12
4	Formal Grammar	13
4.1	Primitives	13
4.2	General Grammar	13
4.3	Grammar relating to the electrical System	15
5	Types	16
5.1	Generic Types	16
5.1.1	Boolean	16
5.1.2	Scalar	16
5.1.3	Set	18
5.1.4	String	19
5.1.5	List	20
5.2	Electrical Types	20
5.2.1	Part	20

5.2.2	Pin	21
5.2.3	Connection	22
5.2.4	Net	22
6	Variables	24
6.1	Variable Declaration and Assignment	24
6.2	Mutability	25
6.3	Scope	25
6.4	Examples	25
7	Expressions	26
7.1	Expression Grammar	26
7.2	Operator Precedence	27
7.3	Evaluation Order	27
8	Operators	28
8.1	Built-in Operators	28
8.1.1	Arithmetic	28
8.1.2	Comparison	29
8.1.3	Boolean Logic	29
8.1.4	Collection Operators	30
8.1.5	Set Comparison	31
8.1.6	Conditional Expression	31
8.2	Operator Precedence	32
8.3	User-defined Operators	32
8.3.1	Definition	32
8.3.2	Examples	33
8.3.3	Precedence	33
9	Constants	34
9.1	Built-in Constants	34
9.1.1	Proper Constants	34
9.1.2	Pseudo-Constants	34
9.2	User-defined Constants	35
10	Control Flow	36
10.1	Conditional Execution	36
10.2	For Loops	36
10.3	Goto Statements	37
10.4	End of Program Statements	37
10.5	Output and Logging	37
11	Functions	38
11.1	Built-in Functions	38
11.1.1	Mathematical Functions	38
11.1.2	String Functions	45
11.1.3	List and Set Functions	46

11.1.4	Shared Functions	47
11.1.5	Special Functions	47
11.2	User-defined Functions	48
11.2.1	Function Definition	48
11.2.2	Function Calls	49
11.2.3	Return Values	49
11.2.4	Local Scoping	49
II	Formal Semantics	51
12	Evaluation Model	52
12.1	Operator Precedence	52
12.2	Type Coercion	52
13	Execution Order	53
13.1	Sequential Execution	53
13.2	Declaration Order	53
14	Constraint Resolution	54
14.1	Conflict Resolution	54
15	Type System Rules	55
15.1	Type Categories	55
15.2	Type Safety	55
III	Core Formal Model	56
16	Structure of the Program	57
16.1	Program Entry Point	57
16.2	Global Scope	57
16.3	Execution Model	57
17	Structure of the Electrical System	58
17.1	Electrical Graph	58
17.2	Object Lifecycle	58
18	Structure of the Solver	59
18.1	Constraint Representation	59
18.2	Resolution Algorithm	59
IV	Hardware Model	60
19	Circuit Model	61
19.1	Graph Structure	61
19.2	Hierarchy	61

20 Electrical Objects	62
20.1 Nets	62
20.1.1 Creation	62
20.1.2 Properties	62
20.2 Parts	62
20.2.1 Creation	62
20.2.2 Properties	63
20.3 Pins	63
20.3.1 Creation	63
20.3.2 Reference Syntax	63
20.4 Connections	63
20.4.1 Creation	63
20.5 Models	64
20.5.1 Model Declaration Syntax	64
20.5.2 Built-in Models	64
20.5.3 Custom Model Examples	65
20.6 Relationship between the Objects	65
21 Querying	66
21.1 Output and Input Quantifiers	66
21.2 Net based Querying	66
21.3 Tagged Net based Querying	66
21.4 Geneneral Querying	67
V Physical System	68
22 Physical Constraints	69
22.1 Units and SI Prefixes	69
22.2 Priority System	69
22.3 Proximity Constraint	70
22.4 Creepage Constraint	70
22.5 Layer Constraint	70
22.6 Board Definition	70
22.7 Net Shrink	71
22.8 Plane Creation	71
22.9 Text Annotation	71
22.10Via Fill	72
22.10.1 Pattern Types	72
22.10.2 Examples	73
22.11Connection Current	73
22.12Alignment	73
22.13Connection Matching	73
22.13.1 Properties	73
22.13.2 Value Calculation Methods	74
22.13.3 Matching Objective	76
22.13.4 Example	77

23 Solver Model	78
23.1 Overview	78
23.2 Algorithm Selection	78
23.2.1 Available Algorithms	78
23.2.2 Default Algorithm	86
23.3 Constraint Resolution	86
23.3.1 No Conflicts	86
23.3.2 Conflicts Between Different Priorities	86
23.3.3 Conflicts Between Same Priorities	87
23.4 Layout Generation Pipeline	87
23.4.1 Phase 1: Constraint Collection	87
23.4.2 Phase 2: Board Setup	87
23.4.3 Phase 3: Component Placement	87
23.4.4 Phase 4: Trace Routing	87
23.4.5 Phase 5: Via Placement	88
23.4.6 Phase 6: Plane Generation	88
23.4.7 Phase 7: Design Rule Check	88
23.4.8 Phase 8: File Export	88
23.5 Ordering of Global Set Constant *e	88
24 Flags	89
24.1 Available Flags	89
VI Toolchain	90
25 Compiler and Execution Pipeline	91
25.1 Compilation Phases	91
25.2 Entry Point	91
26 Code Generation	92
26.1 PCB Generation	92
26.2 Schematic Generation	92
26.3 Export	92
27 Standard Library	93
28 External Library	94
29 Debugging and Logging	95
29.1 Console Output	95
29.2 Log File	95
VII Reference	96
30 Exceptions	97
30.1 Type Errors	97

30.2	Value Errors	97
30.3	Reference Errors	97
30.4	Constraint Errors	97
31	Error and Warning Model	98
31.1	Errors	98
31.2	Warnings	98
32	Future Improvements	99
33	Implementations	100
33.1	Reference Implementation	100
34	References	101
34.1	Language Specification	101
34.2	Related Standards	101
A	Standard Libraries	102
A.1	Material Libraries	102
A.1.1	PCB Substrate Material Library	102
A.1.2	PCB Conductor Material Library	102
A.1.3	PCB Surface-finish Material Library	102
A.2	Basic Model Libraries	102
A.2.1	Passive Component Model Library	102
A.2.2	Active Component Model Library	103
A.2.3	Integrated Circuit Component Model Library	103
A.2.4	Board Hardware Component Model Library	103
A.3	Additional Language Libraries	103
A.4	Footprint Libraries	103
B	Examples	104
B.1	LED Circuit	104
B.1.1	Intent	104
B.1.2	EDADSL Code	104
B.1.3	Explanation	104
C	Diagrams	105
C.1	Execution-phase Diagramm	105

Chapter 1

Introduction

1.1 Abstract

EDADSL (Electronic Design Automation Domain Specific Language) is a Domain-specific Language (DSL) specialized for programmatic Electronic Design Automation (EDA) and Hardware-Synthesis. It is based on an Electrical System and a constraint-driven optimisation engine. This Document is a full outline and as of now the authoritative Specification of the EDADSL Language as well as the systems underlying it.

1.2 Reading this Document

This document is organized into the following parts:

- **Part I: Core Language** – Defines the programming language constructs including lexical structure, types, variables, expressions, constants, control flow, and functions.
- **Part II: Formal Semantics** – Describes the evaluation model, execution order, constraint resolution, and type system rules.
- **Part III: Core Formal Model** – Defines the structure of programs, the electrical system, and the solver.
- **Part IV: Hardware Model** – Describes the circuit model, electrical objects, and querying system.
- **Part V: Physical System** – Defines physical constraints, the solver model, and system flags.
- **Part VI: Toolchain** – Describes the compiler pipeline, code generation, standard library, and debugging.
- **Part VII: Reference** – Contains exceptions, error model, future improvements, and references.
- **Appendices** – Contains standard libraries, examples, and diagrams.

Readers new to EDADSL should start with Parts I and IV, which define the language and electrical system. Those interested in physical layout should read Part V. Implementers should consult Parts II and III for formal specifications.

Part I

Core Language

Chapter 2

Design Principles

2.1 Design Goals

EDADSL was designed with the following primary goals:

1. **Programmatic PCB Design** – Enable engineers to describe circuits and layouts programmatically, enabling version control, automation, and reproducibility.
2. **Constraint-Driven Layout** – Allow physical constraints (clearance, creepage, alignment) to be specified declaratively, with the solver handling the complex optimization.
3. **Electrical Awareness** – Maintain a rich electrical model (parts, pins, connections, nets) that understands circuit topology, not just geometry.
4. **KiCad Integration** – Generate industry-standard KiCad files (schematics and PCBs) for further editing and manufacturing.
5. **Readability** – Use clear, prefixed syntax (elec.*, phys.*, syst.*) to make code self-documenting and easy to understand.
6. **Simplicity** – Provide a minimal set of powerful constructs rather than a large number of specialized features.
7. **Accessibility** – Serve both professional engineers and hobbyists with appropriate defaults while supporting advanced features for complex designs.

2.2 Applications and Application-Areas with special Priority during Design

EDADSL is particularly well-suited for:

- **Power Electronics** – Where creepage distances, high-current traces, and thermal management are critical. The constraint system supports isolation requirements between high-voltage and low-voltage sections.
- **High-Speed Digital Design** – Where trace length matching, impedance control, and signal integrity are important. The `phys.connmatch` constraint supports length and impedance matching.

- **Mixed-Signal Designs** – Where analog and digital sections must be carefully isolated. The constraint system supports separation of noisy and sensitive circuits.
- **Prototyping** – Where rapid iteration and parametric design enable quick exploration of design alternatives.
- **Automated Manufacturing** – Where programmatic generation enables batch production and design automation pipelines.

2.3 Design Philosophy

EDADSL follows a declarative-imperative hybrid philosophy:

- **Imperative Circuit Definition** – Components and connections are defined sequentially, allowing natural expression of circuit topology.
- **Declarative Physical Constraints** – Physical requirements are declared as constraints, letting the solver determine optimal placement and routing.
- **Separation of Concerns** – The electrical description (what the circuit does) is separated from the physical description (how it is laid out).
- **Constraint Hierarchy** – A priority system allows designers to express preferences while ensuring critical requirements are met.
- **Global Scope** – All variables and objects are globally accessible, simplifying code structure at the cost of encapsulation.

Chapter 3

Lexical Structure

3.1 Character Set

EDADSL supports the Full Unicode Character-set, but the Language itself only uses ASCII.

3.2 Whitespace

Whitespace is defined using the following Regular Expression:

```
RegEx:
Whitespace          = [ \t\n\r]+
```

3.3 Comments

Comments are defined using the following Regular Expressions:

```
RegEx:
Line_Comments       = "#" [^\n]*
Multiline_Comments  = "#{ " ... " }#"
```

3.4 Identifiers

Identifiers are defined using the following Regular Expressions:

```
RegEx:
id                  = [_a-zA-Z0-9]+
```

3.5 Keywords

The language uses the following Keywords defined using the following Regular Expressions:

```
Generic_Keywords:  start halt if for gotomarker gotojump
                   writetolog mkpcb mkschematic export
                   echo return let skip
```

```
Physical_Keywords:  phys\[a-z]+
                    Known Physical Keywords:
                    phys.prox  phys.creep  phys.layer
                    phys.side  phys.shrinknet  phys.mkplane
                    phys.viafill  phys.connectioncurrent
                    phys.align  phys.connmatch

Electrical_Keywords: elec\[a-z]+
                    Known Electrical Keywords:
                    elec.part  elec.connect  elec.net
                    elec.tag  elec.untag

System_Keywords:    syst\[a-z]+
                    Known System Keywords:
                    syst.flag\[a-z]+
```

Keywords are reserved identifiers and may not be used as variable names, function names, or constant names.

3.6 Literals

For the Lexical Structure of Literals see: Section 5.1 (Types).

3.7 Operators

For the Lexical Structure of built in Operators see: Section 8 (Operators).
User-defined Operators are defined using the following Regular Expressions:

```
User_Operators      = o\.u\[a-zA-Z0-9]+
```

3.8 Delimiters

EDADSL uses the following Delimiters:

```
() [] {} . , : |
```

3.9 Tokenization Rules

The Tokenization follows the following Rules:

The Lexer uses the Longest Match Rule: in Cases where multiple valid tokizations are possible the longest token is chosen.

If Lexeme matches both a Keyword and an Identifier it is classified as a Keyword.

Whitespace and Comments are discarded.

Chapter 4

Formal Grammar

The Formal Grammar of EDADSL is defined using Extended Backus–Naur form, while Primitives are given using Regular Expressions.

4.1 Primitives

The Primitives are defined using Regular Expressions. For more Information on Lexical structure see Part I Chapter 3.

```
idchar  = [_a-zA-Z0-9]
id       = [_a-zA-Z0-9]+
letter  = [a-zA-Z]
```

4.2 General Grammar

```
program          = "start", { statement }

statement        = assignment
                  | flow_control
                  | function
                  | electrical_system_statement

assignment       = variable_assignment
                  | constant_assignment

function         = function_call_built_in
                  | function_call_ud
                  | function_definition

flow_control     = conditional
                  | iterator
                  | jump
                  | halt
                  | noop
```

```

conditional          = "if", boolean, ":", code_block

iterator             = "for", variable, "E", list, ":", code_block

jump                 = ( "gotomarker" | "gotojump" ), id

function_definition = "function", "f.u.", id,
                      "(", [ { id, "," }, id],
                      ")", ":", "{", f_statement,
                      {f_statement}, "}"

f_statement = statement | "return", [data]

function_call_ud      = "f.u.", id, "(", [{ data, "," }, data] ,")"

function_call_builtin = "f.", id, "(", [{ data, "," }, data] ,")"

variable_assignment   = variable, assigner, [";", "M", "=", boolean]

assigner              = "B=", boolean_type
                      | "V=", scalar_type
                      | "L=", list_type
                      | "W=", string_type
                      | "S=", set_type
                      | electrical_system_assigner

constant_assignment = "let", "c.u.", id, assigner,
                      [";", "P", "=", boolean]

halt                  = "halt"
noop                  = "skip"

list                  = "[", [{ data, "," }, data], "]"

variable              = "$", id

code_block            = "{", statement, {statement} "}"

data                  = variable
                      | literal
                      | constant
                      | function_call_builtin
                      | function_call_ud
                      | expression

```

4.3 Grammar relating to the electrical System

```
electrical_system_statement  = make_statement
                              | elec_statement
                              | phys_statement
                              | syst_statement

electrical_system_assigner    = "P=",      part_type
                              | "C=",      connection_type
                              | "T=",      pin_type
                              | "N=",      net_type

make_statement = "make.", ( "pcb" | "schematic" )

elec_statement = "elec.",
  ( "part", part_reference, ":", model_name,
    "(", model_params, ")", ";", "des", "=",
    string_type, ";", "fp", "=", string_type
  | "connect", ( net_type | pin_type ),
    pin_type, { pin_type }
  | "net", string_type, { string_type }
  | "tag", net_type, string_type,
    { string_type }, ["-remove"]

part_reference = string_type |
  "[" , { string_type, "," }, string_type, "]"
model_name     = id
model_params   = [ { id, "=", data, "," }, id, "=", data ]
```


Chapter 5

Types

This Section defines Type Literals using EBNF and RegEx.

5.1 Generic Types

5.1.1 Boolean

Boolean Data Type Definition

The Boolean data type represents logical truth values. It has exactly two possible values: **True** and **False**. Booleans are used in conditional expressions, logical operations, and control flow decisions.

Boolean Data Type Literals

```
boolean_literal = "True" | "False"
```

Boolean Data Type Casting and Coercion

Boolean values can be coerced from other types:

- Scalar values coerce to **False** if zero, **True** otherwise
- Strings coerce to **False** if empty, **True** otherwise
- Lists and Sets coerce to **False** if empty, **True** otherwise
- **None** coerces to **False**

Explicit conversion is not supported; boolean evaluation is implicit in conditional contexts.

5.1.2 Scalar

Scalar Data Type Definition

The Scalar data type represents real numbers (elements of $\mathbb{R}$). Scalars support standard arithmetic operations and can express values from very small (quecto-scale, 10^{-30}) to very large (quetta-scale, 10^{30}) using SI decimal prefixes.

Scalar Data Type Literals

Scalar literals can be expressed in multiple formats:

EBNF:

```
scalar_literal = decimal_literal | binary_literal
                | octal_literal | hex_literal
```

```
decimal_literal = prefixless_decimal | prefixed_decimal
```

```
prefixless_decimal = ["+" | "-"], digit, {digit},
                    [".", {digit}]
                    | ["+" | "-"], ".", digit, {digit}
                    | ["+" | "-"], digit, {digit},
                    ("e" | "E"), ["+" | "-"], digit,
                    {digit}
```

```
prefixed_decimal = prefix, digit, {digit}
prefix           = "0b" | "0o" | "0d" | "0x"
```

```
binary_literal  = "0b", bindigit, {bindigit}
```

```
octal_literal   = "0o", octdigit, {octdigit}
```

```
hex_literal     = "0x", hexdigit, {hexdigit}
```

RegEx:

```
digit      = [0-9]
bindigit   = [01]
octdigit   = [0-7]
hexdigit   = [0-9a-fA-F]
```

Additionally, SI decimal prefixes may be appended to any numeric literal:

SI_Prefixes:

q	1e-30	quecto	Q	1e+30	quetta
r	1e-27	ronto	R	1e+27	ronna
y	1e-24	yocto	Y	1e+24	yotta
z	1e-21	zepto	Z	1e+21	zetta
a	1e-18	atto	E	1e+18	exa
f	1e-15	femto	P	1e+15	peta
p	1e-12	pico	T	1e+12	tera
n	1e-9	nano	G	1e+9	giga
u	1e-6	micro	M	1e+6	mega
m	1e-3	milli	k	1e+3	kilo
c	1e-2	centi			
d	1e-1	deci			

Examples:

```
0      # = 0
700    # = 700
```

-35	# = -35
.4	# = 0.4
+80	# = 80
6.94	# = 6.94
2e-6	# = 0.000002
-5e9	# = -5000000000
100n	# = 0.0000001
6.2k	# = 6200
4Q	# = 4e30
0x10	# = 16
0b1010	# = 10
0o17	# = 15

Scalar Data Type Casting and Coercion

Scalars can be coerced from other types:

- Booleans coerce to 0 (False) or 1 (True)
- Numeric strings are parsed as scalars (e.g., "42" becomes 42)
- Non-numeric strings raise a type error

Lists of scalars can be passed to mathematical functions, which operate element-wise.

5.1.3 Set

Set Data Type Definition

A Set is an unordered collection of unique elements. Sets can contain any data type:

- Scalars (numbers)
- Strings
- Booleans
- Lists
- Parts
- Connections
- Pins
- Nets

Sets containing mixed types are allowed. Duplicate elements are automatically removed. Sets support standard set algebra operations (union, intersection, difference, subset, element-of).

Set Data Type Literals

EBNF:

```
set_literal = "{", [ { data, "," }, data ], "}"
```

Examples:

```
$f.numbers [set]= { 1, 2, 3, 4, 5 }
$f.mixed [set]= { 1, "hello", True, Q1 }
$f.parts [set]= { Q1, Q2, R3 }
```

Set Data Type Casting and Coercion

Sets can be created from Lists using the `f.set()` function. When converting a List to a Set, duplicate elements are removed and order is lost.

Example:

```
$f.my_list [list]= [ 1, 2, 2, 3, 3, 3 ]
$f.my_set [set]= f.set( $my_list ) # { 1, 2, 3 }
```

Sets can be converted to Lists using the `f.list()` function. The order of elements in the resulting List is determined by the order they appear in the global set `*e`.

5.1.4 String

String Data Type Definition

The String data type represents textual data. Strings can be delimited by single or double quotes, and support escape sequences. Long strings (multi-line) are delimited by triple quotes.

String Data Type Literals

EBNF:

```
string_literal = long_string | short_string
short_string   = "'", {short_string_item_I}, "'" |
                '"', {short_string_item_II}, '"'
long_string    = '"""', {long_string_item_I}, '"""' |
                '"""', {long_string_item_II}, '"""'
```

RegEx:

```
short_string_item_I = [^"\\n)]|(\\. )
short_string_item_II = [^'\\n)]|(\\. )
long_string_item_I   = [^"\\]|(\\. )
long_string_item_II  = [^'\\]|(\\. )
```

String Data Type Casting and Coercion

Strings can be created from other types using the `f.str()` function, which concatenates all arguments into a single string.

Example:

```
$f.msg [string]= f.str( "The value is ", 42,
    " and color is ", "red" )
# $msg = "The value is 42 and color is red"
```

Additional string functions include `f.upperc()` (uppercase), `f.lowerc()` (lowercase), and `f.format()` (formatted output).

5.1.5 List

List Data Type Definition

The List data type represents an ordered, indexed collection of objects. Lists support 0-based indexing, slicing, and can contain elements of different types. Lists are mutable and can be dynamically resized.

List Data Type Literals

EBNF:

```
list_literal = "[", [{ data, "," }, data], "]"
```

List Data Type Casting and Coercion

Lists can be created from Sets using the `f.list()` function. The order is determined by the element order in `*e`.

Lists support range generation using the syntax:

```
[start:increment:end]    # explicit increment
[start:end]              # increment defaults to 1
[start|end|count]        # evenly spaced, count elements
```

Examples:

```
$f.r1 [list]= [0:2:10]    # [0, 2, 4, 6, 8, 10]
$f.r2 [list]= [0:5]      # [0, 1, 2, 3, 4, 5]
$f.r3 [list]= [0|1|5]    # [0, 0.25, 0.5, 0.75, 1]
```

5.2 Electrical Types

5.2.1 Part

Part Data Type Definition

A Part represents a physical electronic component in the circuit. Each Part has a reference designator, a model type, a description, and a footprint. Parts contain Pins which can be connected to Nets or other Pins.

Part Data Type Literals

Parts are created using the `elec.part` statement:

EBNF:

```
part_statement = "elec.part", part_reference,  
    ":", model_name, "(", [model_params], ")",  
    ";", "des", "=", string_type,  
    ";", "fp", "=", string_type
```

```
part_reference = string_type |  
    "[", { string_type, "," }, string_type, "]"  
model_name      = id  
model_params    = [ { id, "=", data, "," },  
                    id, "=", data ]
```

Examples:

```
elec.part "R1": r( R = 100, P = 0.25,  
    type = "metal film" )  
    ; des = "100 ohm resistor"  
    ; fp = "Resistor_SMD:R_0805_2012Metric"
```

```
elec.part ["R2", "R3"]: r( R = 220, P = 0.25 )  
    ; des = "220 ohm resistors"; fp = "Resistor_SMD:R_0805_2012Metric"
```

Part Data Type Casting and Coercion

Parts cannot be cast from other types. Parts are created via `elec.part` statements and accessed by their reference designator. Parts support the following properties:

- Model parameters (accessed via model-specific syntax)
- Pin references (using `part_ref[pin_ref]` syntax)
- Description (set via `des` parameter)
- Footprint (set via `fp` parameter)

5.2.2 Pin

Pin Data Type Definition

A Pin represents a connection point on a Part. Pins are referenced using the syntax `part_ref[pin_ref]`, where `pin_ref` is typically a numeric index or a named terminal (e.g., `R1[1]`, `Q1[d]`, `U1[v+]`).

Pin Data Type Literals

Pins are created implicitly when Parts are instantiated. Pin references are formed using the bracket notation:

```
pin_literal = part_ref, "[", pin_ref, "]"
```

Examples:

```

R1[1]      # Pin 1 of resistor R1
Q1[d]      # Drain pin of MOSFET Q1
U1[v+]     # Non-inverting input of op-amp U1
J1[3]      # Pin 3 of connector J1

```

The available pin references depend on the Part's footprint and model definition.

Pin Data Type Casting and Coercion

Pins cannot be cast from other types. Pins are accessed via Part references and support the following operations:

- Connection to other Pins or Nets via `elec.connect`
- Membership testing in Sets
- Iteration over Pins of a Part

5.2.3 Connection

Connection Data Type Definition

A Connection represents an electrical connection between two Pins. Connections are binary (exactly two Pins) and are created using the pipe syntax or the `elec.connect` statement.

Connection Data Type Literals

Connections can be created using the pipe syntax:

```
connection_literal = "(", pin, "|", pin, ")"
```

Example:

```
( R2[1] | J4[7] ) # R2 pin 1 to J4 pin 7
```

Connections are also created implicitly by `elec.connect` statements.

Connection Data Type Casting and Coercion

Connections cannot be cast from other types. Connections support:

- Membership testing in Sets
- Property queries (length, resistance, inductance, capacitance, impedance)

5.2.4 Net

Net Data Type Definition

A Net represents a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism in the electrical system, allowing multiple components to share a common electrical connection.

Net Data Type Literals

Nets are created using the `elec.net` statement:

EBNF:

```
net_statement = "elec.net", string_type, { string_type }
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

The first argument is the net name; subsequent arguments are optional tags.

Net Data Type Casting and Coercion

Nets cannot be cast from other types. Nets support:

- Tagging/untagging via `elec.tag` and `elec.untag`
- Membership testing in Sets
- Connection to Pins via `elec.connect`
- Querying connected objects using quantifier syntax (e.g., `I@Gnd`, `P@Vin`)

Chapter 6

Variables

Variables in EDADSL are prefixed with the `$` character and are assigned values using type-specific assignment operators.

6.1 Variable Declaration and Assignment

Variables are declared and assigned simultaneously using the following syntax:

EBNF:

```
variable          = "$", id
variable_assignment = variable, "[", type, "]", "=",
                        data, [";", "M", "=", boolean]

type              = "bool" | "scalar" | "list" | "string"
                  | "set" | "part" | "connection" | "pin"
                  | "net"
```

The type determines the variable's data type:

- `bool` – Boolean
- `scalar` – Scalar
- `list` – List
- `string` – String
- `set` – Set
- `part` – Part
- `connection` – Connection
- `pin` – Pin
- `net` – Net

6.2 Mutability

By default, variables are mutable (can be reassigned). To make a variable immutable, append `; M = False` after the assignment:

```
$f.immutable [scalar]= 42; M = False
$f.mutable [scalar]= 100      # Can be reassigned
```

6.3 Scope

Variables have global scope once declared. They can be accessed from any point in the program after their declaration. There is no block scoping for `if` or `for` blocks; variables declared inside these remain accessible outside. However, variables declared inside functions are local to that function and shadow any global variables of the same name (see Section 11.2.4).

6.4 Examples

```
$a [scalar]= 3
$b [scalar]= 5
$f.result [scalar]= $a + $b      # Scalar assignment

$f.name [string]= "EDADSL"      # String assignment
$f.flag [bool]= True           # Boolean assignment

$f.colors [list]= ["red", "green", "blue"] # List
$f.hwset [set]= { Q1, Q2, R5 }   # Set
```

Chapter 7

Expressions

Expressions in EDADSL evaluate to values and can be composed of literals, variables, function calls, and operators.

7.1 Expression Grammar

EBNF:

```
expression      = arithmetic_expression
                  | boolean_expression
                  | comparison_expression
                  | set_expression
                  | function_call
                  | "(" expression ")"
```

```
arithmetic_expression = expression,
    ("+" | "-" | "*" | "/" | "%" | "^"),
    expression
    | "-" expression
```

```
boolean_expression = expression,
    ("&&" | "||" | "^^"), expression
    | "!", expression
```

```
comparison_expression = expression,
   ("<" | "<=" | "==" | "!=" | ">=" | ">"),
    expression
```

```
set_expression = expression, ("u" | "n" | "/"),
    expression
    | "||", expression
    | expression, ("c" | "E" | "!E"), expression
```

7.2 Operator Precedence

Operators are evaluated in the following order of precedence (highest to lowest):

1. Exponentiation ($\wedge$) – right-associative
2. Boolean negation ($!$)
3. Set cardinality ($||a$)
4. Multiplication ($*$), Division ($/$), Modulo ($\%$)
5. Addition ($+$), Subtraction ($-$)
6. Comparison operators ($<$, $<=$, $==$, $!=$, $>=$, $>$)
7. Set comparison (c , E , $!E$)
8. Logical AND ($\&\&$)
9. Logical OR ($||$)
10. Logical XOR ($\wedge\wedge$)
11. Conditional expression ($?->\dots!\dots$)
12. User-defined operators ($o.u.*$)

Parentheses override precedence. See Section 8 for detailed operator descriptions.

7.3 Evaluation Order

Expressions are evaluated left-to-right for operators of equal precedence. Parentheses can be used to override precedence. Function arguments are evaluated before the function is called.

Chapter 8

Operators

8.1 Built-in Operators

8.1.1 Arithmetic

Arithmetic operators perform mathematical operations. Most operators support multiple type combinations.

```
n + m  Addition / Concatenation
n - m  Subtraction
n * m  Multiplication
n / m  Division
n % m  Modulo
n ^ m  Exponentiation
```

Type Overloading

- `+: scalar + scalar → scalar`
- `+: string + string → string (concatenation)`
- `+: list + list → list (append)`
- `-: scalar - scalar → scalar`
- `*: scalar * scalar → scalar`
- `*: scalar * string → string (repetition)`
- `*: scalar * list → list (replication)`
- `/: scalar / scalar → scalar`
- `?: scalar % scalar → scalar`
- `^: scalar ^ scalar → scalar`

Division by zero raises a runtime error. Modulo uses truncated division (`sign(result) = sign(dividend)`). Exponentiation is right-associative: `-2^3 = -8`.

Examples:

```

# Scalar arithmetic
3 + 4      # = 7
10 - 3     # = 7
4 * 5      # = 20
15 / 4     # = 3.75
17 % 5     # = 2
2 ^ 10     # = 1024

# String concatenation
"hello" + " " + "world" # = "hello world"

# String repetition
"ab" * 3      # = "ababab"

# List operations
[1,2] + [3,4]  # = [1,2,3,4]
[0] * 5        # = [0,0,0,0,0]

```

8.1.2 Comparison

Comparison operators evaluate to True or False.

```

n <  m Less Than
n <= m Less or Equal
b == c Equal
b != c Not Equal
n >= m Greater or Equal
n >  m Greater Than

```

Type Overloading

- `<, <=, >=, >`: `scalar` $\rightarrow$ `bool` (ordered)
- `<, <=, >=, >`: `string` $\rightarrow$ `bool` (lexicographic)
- `==, !=`: `any` $\rightarrow$ `bool` (equality)

Ordered comparisons (`<`, `<=`, `>=`, `>`) only work on scalars and strings. Equality (`==`, `!=`) works on any type including lists, sets, parts, and connections.

Examples:

```

3 < 5      # = True
"a" < "b"  # = True (lexicographic)
[1,2] == [1,2] # = True (list equality)
Q1 == Q1    # = True (part identity)

```

8.1.3 Boolean Logic

Boolean operators combine or negate boolean values.

```
b && c Logical And
b || c Logical Or
b ^^ c Logical Xor
!b      Negation
```

- Operands: bool, bool
- Result: bool
- Short-circuit evaluation: a && b evaluates b only if a is True

Examples:

```
True && False    # = False
True || False    # = True
True ^^ False    # = True
True ^^ True     # = False
!True           # = False
```

8.1.4 Collection Operators

Collection operators manipulate sets, lists, and strings.

```
a u b Union (sets only)
a n b Intersection (sets only)
a / b Difference (sets) / Division (scalars)
||a Cardinality / Length
```

Type Overloading

- u: set + set → set (union)
- n: set + set → set (intersection)
- /: set / set → set (difference)
- /: scalar / scalar → scalar (division)
- ||: ||set → scalar (cardinality)
- ||: ||list → scalar (length)
- ||: ||string → scalar (length)

Set operations (u, n) only work on sets. The / operator is overloaded: on sets it returns difference, on scalars it returns division. Cardinality (||) returns the element count for sets, lists, and strings.

Examples:

```
# Set operations
{1,2} u {2,3}    # = {1,2,3} (union)
{1,2} n {2,3}    # = {2} (intersection)
{1,2,3} / {2}    # = {1,3} (difference)
```

```
# Cardinality
||{1,2,3}      # = 3 (set size)
||[1,2,3,4]    # = 4 (list length)
||"hello"      # = 5 (string length)
```

8.1.5 Set Comparison

Set comparison operators test membership and subset relationships.

```
a c b  Subset of
a E b  Element of
a !E b Not element of
```

- Operands: **set**, **set** (subset)
- Operands: any, **set** (membership)
- Result: **bool**
- `a c b` is true if every element of `a` is in `b`
- `a E b` is true if `a` is an element of set `b`

Examples:

```
{1,2} c {1,2,3}    # = True (subset)
1 E {1,2,3}        # = True (membership)
Q1 E *p            # = True (part in all parts)
4 !E {1,2,3}       # = True (not element)
```

8.1.6 Conditional Expression

A ternary-like operator for inline conditional values.

```
(condition)?->(value_if_true)!(value_if_false)
```

- Operands: **bool**, any, any
- Result: type of the selected branch
- Evaluates condition, returns first value if **True**, second if **False**
- Both branches must be of the same type (or coercible)

Examples:

```
$f.x [scalar]= 5
$f.result [scalar]= ($x > 3)?->("big")!("small")
# $result = "big"

$f.val [scalar]= 10
$f.abs [scalar]= ($val >= 0)?->($val)!($val * -1)
```



```
# $abs = 10

$f.a [scalar]= 1
$f.b [scalar]= 2
$f.max [scalar]= ($a > $b)?->($a)!($b)
# $max = 2
```

8.2 Operator Precedence

Operators are evaluated in the following order (highest to lowest):

1. $\wedge$ – Exponentiation (right-associative)
2. $!$ – Boolean negation
3. $||a$ – Cardinality / length
4. $*$, $/$, $\%$ – Multiplication, division, modulo
5. $+$, $-$ – Addition, subtraction
6. $<$, $<=$, $==$, $!=$, $>=$, $>$ – Comparison
7. c , E , $!E$ – Set comparison
8. $\&\&$ – Logical AND
9. $||$ – Logical OR
10. $\wedge$ – Logical XOR
11. $?-> \dots ! \dots$ – Conditional expression
12. User-defined operators ($o.u.*$)

Note: $||$ is unary for cardinality/length (high precedence) but binary for logical OR (low precedence). Parentheses override precedence. Function arguments are evaluated before the function is called.

8.3 User-defined Operators

User-defined operators extend the language with custom infix notation.

8.3.1 Definition

EBNF:

```
"letop", "o.u.", id, assigner, data
```

The operator name must follow the pattern $o.u. [_a-zA-Z0-9]^+$.

8.3.2 Examples

```
letop o.u.mod_minus_1 [scalar]=  
    a % b - 1
```

```
print( 4 o.u.mod_minus_1 3 )  # = 0
```

```
letop o.u.parallel [scalar]=  
    (a * b) / (a + b)
```

```
print( 100 o.u.parallel 100 )  # = 50
```

8.3.3 Precedence

User-defined operators have the lowest precedence among all operators. Use parentheses to ensure correct evaluation order.

Chapter 9

Constants

Constants are immutable values that cannot be reassigned after declaration.

9.1 Built-in Constants

9.1.1 Proper Constants

Proper constants are fixed values provided by the language.

Mathematical Constants

- `c.pi` – The mathematical constant $\pi \approx 3.14159265\dots$
- `c.e` – Euler’s number $e \approx 2.71828182\dots$
- `c.inf` – Positive infinity

Physical Constants

- `c.eps0` – Vacuum permittivity $\varepsilon_0 \approx 8.854 \times 10^{-12}$ F/m
- `c.mu0` – Vacuum permeability $\mu_0 \approx 4\pi \times 10^{-7}$ H/m

9.1.2 Pseudo-Constants

Pseudo-constants are system-defined sets that provide access to electrical objects.

Set Pseudo-Constants

- `*n` – Empty set
- `*e` – All parts, connections, pins, holes, and nets
- `*i` – All pins and connections
- `*u` – All parts, connections, and pins on the upper side of the board
- `*b` – All parts, connections, and pins on the bottom side of the board
- `*h` – All holes

- `*c` – All connections
- `*p` – All parts
- `*t` – All pins

9.2 User-defined Constants

User-defined constants are created using the `let` keyword with the `c.u.` prefix:

EBNF:

```
constant_assignment = "let", "c.u.", id, assigner,
                      [";", "P", "=", boolean]
```

Examples:

```
let c.u.$max_current [scalar]= 30
let c.u.$board_name [string]= "PowerSupply"
```

Constants are immutable by definition and cannot be reassigned.

Chapter 10

Control Flow

EDADSL provides several control flow constructs for conditional execution, iteration, and program flow control.

10.1 Conditional Execution

Conditional execution uses the `if` keyword followed by a boolean expression and a code block:

EBNF:

```
conditional = "if", boolean_expression,  
             ":", code_block
```

Example:

```
$flag [bool]= True  
if $flag ::{  
    f.print("Flag is set")  
}
```

The code block is executed only if the boolean expression evaluates to `True`.

10.2 For Loops

For loops iterate over a list or set:

EBNF:

```
iterator = "for", variable, "E",  
          (list | set), ":", code_block
```

Example:

```
$colors [list]= ["red", "green", "blue"]  
for $c E $colors ::{  
    f.print($c)  
}
```

The loop variable (`$c`) takes on each value in the list/set in sequence.

10.3 Goto Statements

Goto statements provide unconditional jumps to labeled positions:

EBNF:

```
gotomarker = "gotomarker", id
gotojump   = "gotojump", id
```

Example:

```
gotojump skip_section
f.print("This will not be printed")
gotomarker skip_section
f.print("Jumped here")
```

Gotomarkers and gotojumps can cross function and loop boundaries. Nesting is allowed.

10.4 End of Program Statements

EBNF:

```
halt = "halt"
noop = "skip"
```

- **halt** – Terminates program execution and exports all created files
- **skip** – No operation; program continues to next statement

10.5 Output and Logging

EBNF:

```
echo_statement = "echo", expression
writetolog      = "writetolog", string_expression
```

Example:

```
f.print("Hello, EDADSL!")
f.writetolog "Board design completed successfully"
```

- **echo** – Prints a value to the console
- **writetolog** – Appends a message to the log file

Chapter 11

Functions

11.1 Built-in Functions

11.1.1 Mathematical Functions

Absolute Value Function

The Absolute Value Function (denoted as `abs`) returns the Absolute Value of the Argument or the Magnitude of a Vector.

Morphology of the Absolute Value Function:

EBNF:

```
"abs", "(", argument, [ ",", "vec", "=",  
    boolean_data_type ], ")"
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
abs(-3.2)
```

This Example would return a value of 3.2.

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

```
abs([-3, 0.2, 0, -7])
```

This Example would return [3, 0.2, 0, 7].

If `vec = True` is passed a List shall be passed as the Argument and the Function will return Magnitude of the List as an n-dimensional Vector, where n is the Length of the List.

Example:

```
abs( [ 3, -4 ], vec = True )
```

This Example would return 5.0.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Signum Function

The Signum Function (denoted as `sign`) returns the Sign of the Argument.

Morphology of the Signum Function:

EBNF:

```
"sign", "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
sign(-3.2)
```

This Example would return a value of -1 .

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

```
sign([-3, 0.2, 0, -7])
```

This Example would return $[-1, 1, 0, -1]$.

There are no Domain-Restrictions for the function inside $\mathbb{R}^n$.

Regular Trigonometric Functions

Important: Angles are interpreted as Radians by Default.

The Regular Trigonometric Functions (Sine, Cosine and Tangent) (denoted as `sin`, `cos`, `tan`) return the Sine, Cosine, or Tangent of the Argument.

Morphology of the Trigonometric Functions:

EBNF:

```
( "sin" | "cos" | "tan" ), "(", argument,  
  [ ",", "unit", "=", unit ], ")"
```

```
unit = ( "'", ("rad" | "deg" ), "'" )  
       | ( '""', ("rad" | "deg" ), '""' )
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
sin(1.570796327)
```

This Example would return a value of around 1 .

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

```
sin([0, 30, 90], unit = "deg" )
```

This Example would return $[0, 0.5, 1]$.

For Sine and Cosine there are no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

For Tangent there are Domain-Restrictions inside $\mathbb{R}^n$.

It is only defined for $\{x \in \mathbb{R} : \cos x \neq 0\}^n$

Regular Inverse Trigonometric Functions

Important: Angles are interpreted as Radians by Default.

The Regular Inverse Trigonometric Functions (Arcsine, Arccosine and Arctangent) (denoted as `asin`, `acos`, `atan`, `arsin`, `arcos`, `artan` or `arcsin`, `arccos`, `arctan`) return the Inverse for the Sine, Cosine, or Tangent of the Argument.

Morphology of the Trigonometric Functions:

EBNF:

```
inverses_trig_functions, "(" , argument,
  [ ",", "unit", "=", unit ], ")"
```

```
inverses_trig_functions =
  "asin" | "acos" | "atan"
  | "arsin" | "arcos" | "artan"
  | "arcsin" | "arccos" | "arctan"
```

```
unit = ( "'", ("rad" | "deg" ), "'" )
       | ( "'", ("rad" | "deg" ), "' ' )
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
asin(1)
```

This Example would return a value of around 1.570796327.

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

```
arcsin([0, 0.5, 1], unit = "deg" )
```

This Example would return [0, 30, 90].

For Arctangent there is no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

For Arcsine and Arccosine the Domain is Restricted to $[0, 1]^n$.

Inverse Tangent Function based on a Vector Argument

Hyperbolic Functions

The Hyperbolic Functions (Hyperbolic Sine, Hyperbolic Cosine and Hyperbolic Tangent) (denoted as `sinh`, `cosh`, `tanh`) return the Hyperbolic Sine, Hyperbolic Cosine, or Hyperbolic Tangent of the Argument.

Morphology of the Hyperbolic Functions:

EBNF:

```
( "sinh" | "cosh" | "tanh" ), "(", argument, ")"
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
sinh(1.443635475)
```

This Example would return a value of around 2.

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

```
sinh([0, 1, 2] )
```

This Example would return around [0, 1.175, 3.627].

There are no Domain-Restrictions for the functions inside $\mathbb{R}^n$.

Hyperbolic Inverse Functions

The Hyperbolic Inverse Functions (Hyperbolic Arcsine, Hyperbolic Arccosine and Hyperbolic Arctangent) (denoted as `asinh`, `acosh`, `atanh`, `arsinh`, `arcosh`, `artanh` or `arcsinh`, `arccosh`, `arctanh`) return the Inverse for the Hyperbolic Sine, Hyperbolic Cosine, or Hyperbolic Tangent of the Argument.

Morphology of the Hyperbolic Functions:

EBNF:

```
inverses_hypb_functions, "(", argument, ")"
```

```
inverses_hypb_functions =  
    "asinh" | "acosh" | "atanh"  
    | "arsinh" | "arcosh" | "artanh"  
    | "arcsinh" | "arccosh" | "arctanh"
```

If the Type of the Argument does not match the expected Type (a Scalar Value or a List of Scalar Values) a Value Error is raised.

Example:

```
asinh(1)
```

This Example would return a value of around 0.881373587.

The Argument may also be a List of Scalar Values. If This is the case the Function is iterated over each Element.

Example:

`arcsinh( [0, 0.5, 1] )`

This Example would return `[0, 0.481, 0.881]`.

For the Hyperbolic Arcsine there is no Domain-Restrictions for the function inside $\mathbb{R}^n$.

For Hyperbolic Arctangent and Hyperbolic Arccosine the Domain is Restricted to $(-1, 1)^n$ and $[1, \infty)^n$ respectively.

Exponential Functions and Logarithms

Exponential Function The `exp` function returns e raised to the power of the argument.

Morphology:

EBNF:

`"exp", "(", argument, ")"`

Example:

`exp(0)      # Returns 1.0`

`exp(1)      # Returns approximately 2.71828`

Square Root Function The `sqrt` function returns the square root of the argument.

Morphology:

EBNF:

`"sqrt", "(", argument, ")"`

Example:

`sqrt(9)     # Returns 3.0`

`sqrt(2)     # Returns approximately 1.41421`

Domain: $[0, \infty)$

Power Function The `pow` function returns the first argument raised to the power of the second argument.

Morphology:

EBNF:

`"pow", "(", argument, ",", argument, ")"`

Example:

`pow(2, 3)    # Returns 8.0`

`pow(9, 0.5) # Returns 3.0`

Natural Logarithm The `ln` function returns the natural logarithm (base e) of the argument.

Morphology:

EBNF:

`"ln", "(", argument, ")"`

Example:

```
ln(1)      # Returns 0.0
ln(e)      # Returns 1.0
```

Domain: $(0, \infty)$

Base-10 Logarithm The `log` function returns the base-10 logarithm of the argument.
Morphology:

EBNF:

```
"log", "(", argument, ")"
```

Example:

```
log(100)   # Returns 2.0
log(1)     # Returns 0.0
```

Domain: $(0, \infty)$

Rounding Functions

Floor Function The `floor` function rounds the argument down to the nearest integer.
Morphology:

EBNF:

```
"floor", "(", argument, ")"
```

Example:

```
floor(2.7) # Returns 2
floor(-1.3) # Returns -2
```

Ceiling Function The `ceil` function rounds the argument up to the nearest integer.
Morphology:

EBNF:

```
"ceil", "(", argument, ")"
```

Example:

```
ceil(2.3)  # Returns 3
ceil(-2.7) # Returns -2
```

Round Function The `round` function rounds the argument to the nearest integer.
Morphology:

EBNF:

```
"round", "(", argument, ")"
```

Example:

```
round(2.5) # Returns 3
round(2.4) # Returns 2
```

Extrema Functions

Minimum Function The `min` function returns the minimum value in a list.

Morphology:

EBNF:

```
"min", "(", list, ")"
```

Example:

```
min([3, 1, 4, 1, 5])    # Returns 1
```

Maximum Function The `max` function returns the maximum value in a list.

Morphology:

EBNF:

```
"max", "(", list, ")"
```

Example:

```
max([3, 1, 4, 1, 5])    # Returns 5
```

Inverse Tangent Function based on a Vector Argument

The `atan2` function computes the arctangent of y/x using the signs of both arguments to determine the correct quadrant.

Morphology:

EBNF:

```
"atan2", "(", argument, ",", argument, ")"
```

Example:

```
atan2(1, 0)    # Returns approximately 1.5708 (pi/2)
```

```
atan2(0, 1)    # Returns 0.0
```

Norm and Normalization Functions

EDADSL provides functions for computing vector norms and normalizing vectors.

Norm Function The `norm` function computes the p-norm of a vector (list of scalars).

Morphology of the Norm Function:

EBNF:

```
"norm", "(", argument, [ ",", "p", "=",  
    scalar_data_type ], ")"
```

Parameters:

- **argument** – A list of scalar values (the vector)
- **p** – (Optional) The order of the norm. Defaults to 2 (Euclidean norm).

Supported norms:

- $p = 1$ – L1 norm (Manhattan distance): $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$
- $p = 2$ – L2 norm (Euclidean distance): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- $p > 2$ – Lp norm: $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$
- $p = 0$ – L0 "norm" (count of non-zero elements): $\|\mathbf{x}\|_0 = |\{i : x_i \neq 0\}|$
- $p = -1$ – L-infinity norm (Chebyshev distance): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

Examples:

```
norm([3, 4])           # Returns 5.0 (L2 norm)
norm([3, 4], p=1)      # Returns 7 (L1 norm)
norm([3, 4], p=0)      # Returns 2 (L0 norm)
norm([-3, 4, -2], p=-1) # Returns 4 (L-inf norm)
```

If the argument is not a list of scalars, a Value Error is raised.

Normalize Function The `normalize` function returns a unit vector in the direction of the input vector.

Morphology of the Normalize Function:

EBNF:

```
"normalize", "(", argument, [ ",", "p", "=",
                           scalar_data_type ], ")"
```

Parameters:

- **argument** – A list of scalar values (the vector to normalize)
- **p** – (Optional) The order of the norm to use for normalization. Defaults to 2.

The function computes: $\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_p}$

Examples:

```
normalize([3, 4])       # Returns [0.6, 0.8] (L2)
normalize([3, 4], p=1)  # Returns [0.43, 0.57] (L1)
normalize([6, 8], p=2)  # Returns [0.6, 0.8] (L2)
```

If the norm of the vector is zero, a Division by Zero Error is raised.

Relationship between `abs(vec=True)` and `norm` The function `abs(v, vec=True)` is equivalent to `norm(v)` (L2 norm by default).

11.1.2 String Functions

String Conversion

The `str` function converts its arguments to a string.

Morphology:

EBNF:

```
"str", "(", { argument }, ")"
```

Example:

```
str("Hello")          # Returns "Hello"
str(42)                # Returns "42"
str("Value: ", 3.14)   # Returns "Value: 3.14"
```

String Formatting

The `format` function formats a non-string datatype.

Morphology:

EBNF:

```
"format", "(", argument, [ ",", options ], ")"
```

String Case Functions

Uppercase The `upperc` function converts a string to uppercase.

Morphology:

EBNF:

```
"upperc", "(", string, ")"
```

Example:

```
upperc("hello")      # Returns "HELLO"
```

Lowercase The `lowerc` function converts a string to lowercase.

Morphology:

EBNF:

```
"lowerc", "(", string, ")"
```

Example:

```
lowerc("HELLO")      # Returns "hello"
```

11.1.3 List and Set Functions

Concatenation

The `concat` function combines all arguments into a single list.

Morphology:

EBNF:

```
"concat", "(", { argument }, ")"
```

Example:

```
concat([1, 2], [3, 4])      # Returns [1, 2, 3, 4]
concat("a", "b", "c")       # Returns ["a", "b", "c"]
```

Type Conversion

List to Set The `set` function converts a list to a set. Only objects allowed in sets (parts, connections, pins, nets) are preserved; forbidden objects are silently dropped with a warning. Morphology:

EBNF:

```
"set", "(", list, ")"
```

Example:

```
set([Q1, Q2, R5])    # Returns {Q1, Q2, R5}
```

Set to List The `list` function converts a set to a list. The order is determined by the order elements appear in `*e`.

Morphology:

EBNF:

```
"list", "(", set, ")"
```

Example:

```
list({Q1, Q2})    # Returns [Q1, Q2] (order from *e)
```

11.1.4 Shared Functions

Length/Size Function

The `len` function returns the length of a list or string, or the size of a set.

Morphology:

EBNF:

```
"len", "(", ( list | string | set ), ")"
```

Examples:

```
len([1, 2, 3])      # Returns 3
len("hello")        # Returns 5
len({Q1, Q2, R5})   # Returns 3
```

11.1.5 Special Functions

Print Function

The `print` function outputs its argument to the console. Unlike `echo`, it can directly evaluate certain expressions.

Morphology:

EBNF:

```
"print", "(", argument, ")"
```

Example:

```
print("Hello, World!")
print(3 + 2 * 5)    # Outputs: 13
```


Type Function

The `type` function returns the type of an object as a string.
Morphology:

EBNF:

```
"type", "(", argument, ")"
```

Example:

```
type(42)      # Returns "scalar"  
type("hello") # Returns "string"  
type(Q1)      # Returns "part"
```

Reference Function

The `ref` function converts a string containing a part reference into the referenced part.
Morphology:

EBNF:

```
"ref", "(", string, ")"
```

Example:

```
ref("R1")    # Returns the part referenced as R1
```

No-Operation Function

The `noop` function is the identity function. It returns its argument unchanged.
Morphology:

EBNF:

```
"noop", "(", argument, ")"
```

Example:

```
noop(42)      # Returns 42  
noop("hello") # Returns "hello"  
noop(Q1)      # Returns Q1
```

Use `noop` as a placeholder in queries where no transformation is needed.

11.2 User-defined Functions

User-defined functions allow creation of reusable code blocks with local scoping.

11.2.1 Function Definition

Functions are defined using the `function` keyword with the `f.u.` prefix:

EBNF:

```
function_definition = "function", "f.u.", id,  
    "(", [ { id, "," }, id ], ")", ":", "{",  
    f_statement, {f_statement}, "}"
```

```
f_statement = statement | "return", [data]
```

Example:

```
function f.u.add( $a, $b )::{  
    $retval [scalar]= $a + $b  
}
```

Function parameters are local variables scoped to the function body. To return a value, set the `$retval` variable.

11.2.2 Function Calls

User-defined functions are called using the `f.u.` prefix:

EBNF:

```
function_call_ud = "f.u.", id, "(",  
    [{ data, "," }, data ], ")"
```

Example:

```
$f.result [scalar]= f.u.add( 3, 5 )    # $result = 8
```

11.2.3 Return Values

Functions return values by setting the `$retval` variable. If `$retval` is not set, the function returns nothing:

```
function f.u.square( $x )::{  
    $retval [scalar]= $x ^ 2  
}
```

```
function f.u.greet( $name )::{  
    f.print( f.str( "Hello, ", $name, "!" ) )  
    # No return value  
}
```

```
$f.sq [scalar]= f.u.square( 4 )        # $sq = 16  
f.u.greet( "World" )                  # Prints: Hello, World!
```

11.2.4 Local Scoping

Functions have local scoping. Variables declared inside a function are only accessible within that function. Parameters are also local variables.

Scope Rules

- Parameters (\$a, \$b, etc.) are local to the function body
- Variables declared inside the function with `$var [type]= val` are local
- Local variables shadow global variables of the same name
- The `$retval` variable is always local to the function
- Functions cannot access local variables from other function invocations

Examples

```
$f.x [scalar]= 10      # Global variable

function f.u.doublen()::{
    $f.x [scalar]= $f.x * 2  # Local $x shadows global
    f.print($f.x)           # Prints: 20
}

f.u.doublen()
f.print($f.x)             # Prints: 10 (global unchanged)

function f.u.addLocal( $a, $b )::{
    $sum [scalar]= $a + $b   # $sum is local
    $retval [scalar]= $sum
}

f.u.addLocal( 3, 4 )    # Returns 7
# $sum is not accessible here
```

Recursion Local scoping enables recursion since each call has its own copy of parameters:

```
function f.u.factorial( $n )::{
    if $n <= 1 ::{
        $retval [scalar]= 1
    } else ::{
        $retval [scalar]= $n *
            f.u.factorial( $n - 1 )
    }
}

$f.result [scalar]= f.u.factorial( 5 )  # = 120
```

Part II

Formal Semantics

Chapter 12

Evaluation Model

Expressions in EDADSL are evaluated according to standard mathematical precedence rules. The evaluation model is eager (strict), meaning all sub-expressions are evaluated before the operator is applied.

12.1 Operator Precedence

The complete operator precedence is defined in Section 8.

12.2 Type Coercion

Chapter 13

Execution Order

13.1 Sequential Execution

Program statements are executed sequentially from the entry point (**start**).

13.2 Declaration Order

Chapter 14

Constraint Resolution

Physical constraints are resolved by the solver according to the priority system defined in Section 22.

14.1 Conflict Resolution

When constraints conflict, the solver applies the higher-priority constraint and optimizes the lower-priority one. See Section 23 for the complete resolution rules.

Chapter 15

Type System Rules

EDADSL uses a dynamic type system with distinct types for electrical and non-electrical objects.

15.1 Type Categories

- Generic types: Boolean, Scalar, String, List, Set
- Electrical types: Part, Pin, Connection, Net

15.2 Type Safety

Part III

Core Formal Model

Chapter 16

Structure of the Program

16.1 Program Entry Point

Every EDADSL program must begin with the `start` keyword. The program executes sequentially from the entry point.

16.2 Global Scope

Variables and constants declared at the top level of the program operate on the global scope. They can be accessed from any point in the program after their declaration. However, variables declared inside functions are local to that function (see Section 11.2.4).

16.3 Execution Model

Chapter 17

Structure of the Electrical System

17.1 Electrical Graph

The electrical system is modeled as an undirected graph where:

- Vertices represent Nets
- Edges represent Connections
- Parts attach to the graph via their Pins

17.2 Object Lifecycle

Chapter 18

Structure of the Solver

18.1 Constraint Representation

Physical constraints are represented as optimization objectives with priorities.

18.2 Resolution Algorithm

Part IV

Hardware Model

Chapter 19

Circuit Model

The circuit model represents the electrical system as a graph of interconnected objects.

19.1 Graph Structure

The circuit is represented as a graph where:

- Nodes are Nets (electrical connection points)
- Edges are Connections (physical wires/traces between pins)
- Parts are components that connect to the graph via Pins

19.2 Hierarchy

The model supports hierarchical grouping:

- Parts contain Pins
- Pins connect to Nets via Connections
- Nets can be grouped by Tags

Chapter 20

Electrical Objects

The electrical system consists of five primary object types.

20.1 Nets

A Net is a named electrical node that connects multiple Pins and Connections. Nets are the fundamental grouping mechanism for shared electrical connections.

20.1.1 Creation

Nets are created using the `elec.net` statement:

```
elec.net <name> [<tag1> ... <tag n>]
```

Example:

```
elec.net "Gnd"
```

```
elec.net "hvbus" "hv" "high_i" "noisy"
```

20.1.2 Properties

- Name (unique identifier)
- Connected objects (Pins and Connections)
- Tags (user-defined labels)

20.2 Parts

A Part represents a physical electronic component in the circuit.

20.2.1 Creation

Parts are created using the `elec.part` statement:

```
elec.part <reference>: <model>(<args>);
    des="<description>"; fp="<footprint>"
```

Example:

```
elec.part "R3": R(R=220, P=0.25, type="carbon");
    des="220 ohm resistor";
    fp="Resistor_THT:R_Axial_DIN0309"
```

20.2.2 Properties

- Reference designator (e.g., R3, Q1, U1)
- Model and parameters
- Description
- Footprint
- Pins (derived from model/footprint)

20.3 Pins

A Pin is a connection point on a Part. Pins are accessed using the bracket notation.

20.3.1 Creation

Pins are created implicitly when Parts are instantiated.

20.3.2 Reference Syntax

```
<part_ref>[<pin_ref>]
```

Examples:

```
R1[1]    # Pin 1 of resistor R1
Q1[d]    # Drain pin of MOSFET Q1
U1[v+]   # Non-inverting input of op-amp U1
```

20.4 Connections

A Connection is a binary electrical connection between exactly two Pins.

20.4.1 Creation

Connections can be created using the pipe syntax or `elec.connect`:

```
# Pipe syntax (creates a Connection object):
( R2[1] | J4[7] )

# Connect statement (joins pins to a Net):
```



```
elec.connect <net> <pin1> <pin2> ... <pin n>
```

Example:

```
elec.connect Gnd Q2[s] C6[-] R3[2]
```

20.5 Models

Models define the electrical characteristics and available pins of a Part type.

20.5.1 Model Declaration Syntax

Custom models are defined using the following syntax:

EBNF:

```
model_declaration = "model", model_name, ":",  
                    "{", pin_entries, [metadata], "}"
```

```
model_name        = id, "/", id
```

```
pin_entry         = pin_name, "->", pin_id, ";",  
                    "T", "=", pin_type
```

```
pin_name          = id
```

```
pin_id            = integer
```

```
pin_type          = "input" | "power" | "output"
```

```
metadata          = des_entry | dts_entry | fp_entry
```

```
des_entry         = "des", "=", string_type
```

```
dts_entry         = "dts", "=", string_type
```

```
fp_entry          = "fp", "=", string_type
```

The `model_name` consists of a category and variant separated by / (e.g., `mosfet/C3M0016120K1`).

Pin types classify the electrical role:

- **input** – Signal input (gate, base, control)
- **power** – Power connection (source, drain, emitter, collector)
- **output** – Signal output (drain for sense, kelvin connections)

20.5.2 Built-in Models

- **R** – Resistor
- **C** – Capacitor
- **L** – Inductor
- **opamp** – Operational Amplifier

- MOSFET – MOSFET transistor
- Additional models available in standard library

20.5.3 Custom Model Examples

MOSFET Model

```
model MOSFET/IRLZ44N:{
    g -> 1; T = input
    S -> 2; T = power
    d -> 3; T = power
}
```

Non-Polarized Capacitor

```
model nonpol_C:{
    1 -> 1; T = power
    2 -> 2; T = power
}
```

SiC MOSFET with Metadata

```
model mosfet/C3M0016120K1:{
pins:
    g      -> 4; T = input
    s      -> 2; T = power
    d      -> 1; T = power
    kelvin_s -> 3; T = output

des: "1200 V, 125A, 16 mOhm, T0-247-4"

dts: "https://assets.wolfspeed.com/..."

fp:  "Package_T0_S0T_THT/T0-247-4_Vert"
}
```

The **des** (description), **dts** (datasheet URL), and **fp** (footprint path) are optional meta-data entries.

20.6 Relationship between the Objects

- Parts contain Pins
- Pins are connected to Nets via Connections
- Nets can have Tags for grouping
- The global set ***e** contains all objects in creation order

Chapter 21

Querying

EDADSL provides a powerful querying system for selecting electrical objects based on their relationships.

21.1 Output and Input Quantifiers

The quantifier syntax selects objects connected to a net:

```
<quantifier>@<net_name>
```

21.2 Net based Querying

Quantifiers filter by object type:

```
P@<net>    # Parts connected to net
C@<net>    # Connections on net
T@<net>    # Pins connected to net
I@<net>    # Connections and Pins on net
H@<net>    # Holes on net
E@<net>    # Everything on net (or just @<net>)
```

Example:

```
$input S= I@Vin
```

21.3 Tagged Net based Querying

Query by tag instead of net name:

```
<quantifier>@^<tag>
```

This expands to the union of the quantifier across all nets with the given tag.

Example:

```
tag Vin "sensitive"
tag Vfb "sensitive"
I@^sensitive # All connections/pins on
              # nets tagged "sensitive"
```

21.4 General Querying

General querying provides two powerful forms for filtering and transforming sets: set builder notation and set query notation. Both support nesting.

21.4.1 Set Builder Notation

Set builder notation iterates over a set or list, filtering and optionally transforming elements.

EBNF:

```
set_builder = "{", variable, "E", ( set | list ),
              ":", filter_expr, [ "~", transform_expr ], "}"
```

```
filter_expr  = function_call    # returns bool
transform_expr = function_call  # applied to filtered elements
```

The filter function receives each element and returns `True` to keep it. The transform function is applied to all elements that pass the filter. If transform is omitted, elements are returned as-is.

Examples:

```
# Filter: only active parts
{ $x E *p : f.u.is_active($x) ~ noop($x) }

# Filter and transform: get names of large caps
{ $x E *p : f.u.is_capacitor($x) &&
          f.u.value($x) > 100u ~
          f.u.name($x) }

# Nested: find all nets connected to parts
# in a set
{ $x E *p : f.u.is_mosfet($x) ~
  { $y E *n : f.u.connected($y, $x) ~
    noop($y) } }
```

21.4.2 Set Query Notation

Set query notation applies a filter and optional transform to an existing set, with optional output assignment.

EBNF:

```
set_query = "{", set_expr, "@~", "[",
              filter_expr, "]",
              [ "~[", transform_expr, "]" ],
              ">", [ variable ], "}"
```

The `@~` operator queries the set. The trailing `>` optionally writes the result to a variable; if omitted, the result is returned directly.

Examples:

```

# Query with filter only
{ *p @~[f.u.is_active] ~> }

# Query with filter and transform
{ *p @~[f.u.is_mosfet]
  ~[f.u.name] ~> }

# Store result in variable
{ *p @~[f.u.is_capacitor]
  ~[f.u.value] ~> $cap_values }

# Query connections on a net
{ C@Vin @~[f.u.is_high_current] ~> }

```

21.4.3 Nesting

Both forms can be nested to any depth. Each nested query creates a new scope for its loop variable.

Example:

```

# For each MOSFET, find all connected nets,
# then filter to high-voltage nets
{ $q E *p : f.u.is_mosfet($q) ~
  { C@~power @~[f.u.connected_to($q)]
    ~[f.u.voltage_rating] ~>
    $ratings } }

```

Chapter 22

Physical Constraints

Physical constraints are specified using the `phys.` prefix and control the physical layout and manufacturing requirements of the PCB.

22.1 Units and SI Prefixes

Warning: All physical values use base SI units (meters, ohms, farads, etc.). A value like `min=15` means 15 **meters**, not 15 millimeters!

Always use SI prefixes for typical PCB dimensions:

- `min=15m` – 15 millimeters (15×10^{-3} m)
- `min=15u` – 15 micrometers (15×10^{-6} m)
- `d=0.25m` – 0.25 millimeters (250 μ m)

SI prefixes available: `n` (10^{-9}), `u` (10^{-6}), `m` (10^{-3}), `k` (10^3), `M` (10^6), `G` (10^9). See Section ?? for details.

22.2 Priority System

All physical constraints accept a priority parameter that determines how conflicts are resolved:

Priorities:

<code>(none)</code> or <code>-d</code>	Default
<code>-nth</code>	Nice to have
<code>-imp</code>	Important
<code>-vimp</code>	Very important
<code>-oeo</code>	Only explicit overrides
<code>-eo</code>	Explicit override
<code>-ne</code> = <code>-nonneg</code>	No exceptions

Priority Chain:

`-d` < `-nth` < `-imp` < `-vimp` < `-oeo` < `-eo` < `-ne` = `-nonneg`

22.3 Proximity Constraint

Controls the distance between sets of objects:

```
phys.prox <set_a> <set_b>  
    min=<min_distance> max=<max_distance>  
    <priority>  
phys.prox <set> minim <priority>
```

- **min** – Minimum distance between edges of set a and set b
- **max** – Maximum distance anything in set a can be from anything in set b
- **minim** – Tries to minimize distance between all objects in the set

Example:

```
phys.prox $hvsys $lvsys min=15m -ne  
phys.prox $noisy_loop minim -vimp
```

22.4 Creepage Constraint

Controls the surface distance between sets (affected by slots, vias, traces, and conductive material):

```
phys.creep <set_a> <set_b>  
    d=<min_creepage_distance> <priority>
```

Note: Unlike **phys.prox** (Euclidean distance), **phys.creep** measures surface distance. Conductive material contributes 0 distance.

Example:

```
phys.creep $hvsys $lvsys d=30m -nonneg
```

22.5 Layer Constraint

Assigns objects to specific board layers:

```
phys.layer <set> <layer_number> <priority>
```

Layer 0 is the top layer; the bottom layer is layer n (where n is the total number of layers).

Example:

```
phys.layer *p 0 -oeo
```

22.6 Board Definition

Defines the PCB stackup characteristics. Can only be invoked once:

```
phys.board N=<layers> T=[<thicknesses>]  
          W=[<weights>] M=[<materials>]
```

- N – Number of board layers
- T – List of thicknesses between layers in mm (length $n - 1$)
- W – List of copper weights in oz/ft² (length n)
- M – List: [dielectric, conductor, surface-plating]

Default: `phys.board N=2 T=[1.6] W=[1, 1] M=["FR4", "copper", "HASL"]`

Example:

```
phys.board N=2 T=[1.6] W=[2, 2]
          M=["FR4", "copper", "ENIG"]
```

22.7 Net Shrink

Minimizes the physical extent of a net:

```
phys.shrinknet <net> <priority>
```

Example:

```
net "Vsw"
phys.shrinknet Vsw -vimp
```

22.8 Plane Creation

Creates a copper plane on a layer connected to a net:

```
phys.mkplane <layer_number> <net>
```

Example:

```
net "Gnd"
phys.mkplane 1 Gnd
```

22.9 Text Annotation

Places text on the board as a physical object:

```
phys.mktext txt = <text>
          font = <font_name>
          size = <point_size>
          material = <material>
          layer = <layer>
```

- txt – Text string to display
- font – Font name (e.g., "Ubuntu Mono")
- size – Font size in points

- **material** – Material ("cu" for copper silkscreen)
- **layer** – Target layer (e.g., T for top silkscreen)

Example:

```
phys.mktext txt = "HV" font = "Ubuntu Mono"
    size = 9 material = "cu" layer = T
```

22.10 Via Fill

Fills a pad with vias for thermal management:

```
phys.viafill <pad> v=<via_hole_diameter>
    d=<via_spacing> p=<pattern>
```

- **v** – Via hole diameter (use m for mm)
- **d** – Minimum spacing between via centers
- **p** – Fill pattern (default: **grid**)

22.10.1 Pattern Types

grid Regular rectangular matrix of vias. Simple, predictable routing channels.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=grid
```

hex Hexagonal honeycomb packing. Approximately 15% denser than grid, better thermal performance.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=hex
```

stagger Offset rows forming triangular lattice. Balances density with routing space between rows.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=stagger
```

radial Concentric rings emanating from pad center. Optimal for round thermal pads under ICs.

```
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial
```

random Poisson-disk distributed vias. Avoids periodic structures that can cause resonance or EMI.

```
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.10.2 Examples

```
# Standard grid fill for MOSFET drain
phys.viafill Q1[d] v=0.25m d=0.5m p=grid

# Dense hex fill for high-current pad
phys.viafill Q1[d] v=0.2m d=0.4m p=hex

# Radial fill for IC thermal pad
phys.viafill U1[pin5] v=0.3m d=0.8m p=radial

# Random fill to avoid EMI resonance
phys.viafill Q1[d] v=0.25m d=0.5m p=random
```

22.11 Connection Current

Sets the minimum RMS current a connection must carry:

```
phys.connectioncurrent <connection | set>
    <rms_current> <priority>
```

Example:

```
phys.connectioncurrent ( J1[2] | F1[1] ) 30 -oeo
```

22.12 Alignment

Aligns parts along an axis with specified spacing:

```
phys.align <list_of_parts>
    axis=<x|y> d=<edge_spacing> <priority>
```

Example:

```
phys.align [ Q1, Q2 ] axis=x d=5m -vimp
```

22.13 Connection Matching

Minimizes variance of a property across a set of connections:

```
phys.connmatch <property>
    <set_of_connections> <priority>
```

22.13.1 Properties

- `len` – Physical trace length
- `res` – DC resistance
- `ind` – Self-inductance

- **cap** – Capacitance to reference plane
- **imp** – Characteristic impedance
- **t** – Signal propagation delay

22.13.2 Value Calculation Methods

Calculation Mode Selection

The calculation method can be selected using the `syst.cm.calc` flag:

`syst.cm.calc <mode>`

Available modes:

- **approx** – (Default) Uses simplified closed-form formulas for fast computation. Suitable for most designs.
- **calculus** – Uses numerical integration and differential equations for higher accuracy. Computationally more expensive.
- **hybrid** – Uses closed-form formulas for initial estimates, then refines with calculus-based methods for critical nets.

Example:

```
syst.cm.calc calculus
phys.connmatch imp $high_speed_lines -vimp
```

Length (len)

The physical length of the trace path from source pin to destination pin, measured in millimeters. Calculated as the sum of all trace segments.

Resistance (res)

DC resistance is calculated using:

$$R = \frac{\rho \cdot L}{W \cdot T} \quad (22.1)$$

where:

- ρ – Resistivity of the conductor (copper: $1.72 \times 10^{-8} \Omega \cdot \text{m}$)
- L – Trace length (mm)
- W – Trace width (mm)
- T – Trace thickness (mm, derived from copper weight in oz/ft²)

Calculus Mode When `syst.cm.calc calculus` is active, skin effect is considered:

$$R_{ac} = R_{dc} \cdot \left(1 + \frac{1}{12} \cdot \left(\frac{\delta}{T} \right)^4 \right) \quad (22.2)$$

where $\delta = \sqrt{\frac{2\rho}{\omega\mu}}$ is the skin depth at frequency ω .

Inductance (ind)

Self-inductance is approximated using the loop inductance formula for a microstrip:

$$L \approx \frac{\mu_0 \cdot L_{trace}}{2\pi} \cdot \ln \left(\frac{2h}{w+t} \right) \quad (22.3)$$

where:

- μ_0 – Permeability of free space ($4\pi \times 10^{-7}$ H/m)
- L_{trace} – Trace length
- h – Height above reference plane
- w – Trace width
- t – Trace thickness

Calculus Mode When `syst.cm.calc calculus` is active, mutual inductance and frequency-dependent effects are computed using numerical integration:

$$L_{total} = \int_0^{L_{trace}} \frac{\mu_0}{2\pi} \cdot \ln \left(\frac{2h(x)}{w(x)+t} \right) dx \quad (22.4)$$

This accounts for trace width variations and non-uniform height above the reference plane.

Capacitance (cap)

Capacitance to the reference plane is calculated using the microstrip capacitance formula:

$$C \approx \frac{\varepsilon_0 \cdot \varepsilon_r \cdot w \cdot L_{trace}}{h} \quad (22.5)$$

where:

- ε_0 – Permittivity of free space (8.854×10^{-12} F/m)
- ε_r – Relative permittivity of the dielectric (FR4: ≈ 4.5)
- w – Trace width
- L_{trace} – Trace length
- h – Height above reference plane

Calculus Mode When `syst.cm.calc calculus` is active, fringing effects and dielectric variations are included:

$$C = \int_0^{L_{trace}} \frac{\varepsilon_0 \cdot \varepsilon_r(x) \cdot w_{eff}(x)}{h(x)} dx \quad (22.6)$$

where w_{eff} includes fringing capacitance contributions and $\varepsilon_r(x)$ accounts for material variations along the trace.

Impedance (imp)

Characteristic impedance for a microstrip is calculated as:

$$Z_0 \approx \frac{87}{\sqrt{\varepsilon_r + 1.41}} \cdot \ln \left(\frac{5.98h}{0.8w + t} \right) \quad (22.7)$$

For stripline:

$$Z_0 \approx \frac{60}{\sqrt{\varepsilon_r}} \cdot \ln \left(\frac{4b}{0.67(0.8w + t)} \right) \quad (22.8)$$

where b is the total dielectric thickness for stripline configurations.

Calculus Mode When `syst.cm.calc calculus` is active, the full telegrapher's equations are solved numerically:

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (22.9)$$

where R , L , G , and C are per-unit-length parameters computed from the trace geometry and material properties. This accounts for frequency-dependent losses and dispersion.

Propagation Time (t)

Signal propagation delay is calculated as:

$$t = \frac{L_{trace}}{v_p} = \frac{L_{trace} \cdot \sqrt{\varepsilon_{eff}}}{c} \quad (22.10)$$

where:

- v_p – Propagation velocity
- c – Speed of light in vacuum (3×10^8 m/s)
- ε_{eff} – Effective permittivity (depends on geometry and dielectric)

Calculus Mode When `syst.cm.calc calculus` is active, propagation delay is computed by integrating along the trace path:

$$t = \int_0^{L_{trace}} \frac{\sqrt{\varepsilon_{eff}(x)}}{c} dx \quad (22.11)$$

This accounts for varying dielectric thickness, material transitions, and impedance discontinuities along the trace.

22.13.3 Matching Objective

The solver minimizes the scaled variance of the selected property across all connections in the set:

$$\text{minimize} \quad \frac{\text{var}(P)}{||S||} \quad (22.12)$$

where P is the property value for each connection and $||S||$ is the cardinality of the connection set.

22.13.4 Example

```
phys.connmatch t $CPU_to_RAM_lines -vimp
```

This ensures all CPU-to-RAM signal lines have matched propagation delays, critical for parallel data buses.

Chapter 23

Solver Model

23.1 Overview

The EDADSL solver transforms declarative physical constraints into an optimized PCB layout. The process involves collecting constraints, selecting an optimization algorithm, and generating the final layout through placement and routing phases.

23.2 Algorithm Selection

The solver algorithm can be selected using the `syst.alg` flag:

```
syst.alg <algorithm_name>
```

23.2.1 Available Algorithms

Simulated Annealing (sa)

A probabilistic optimization technique inspired by the annealing process in metallurgy. The algorithm explores the solution space by accepting worse solutions with a probability that decreases over time.

Objective Function Let $\mathbf{x} \in \mathcal{X}$ represent the complete layout state (component positions, trace routes, via placements). The objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ quantifies the total constraint violation cost:

$$f(\mathbf{x}) = \sum_{i=1}^n w_i \cdot v_i(\mathbf{x}) \quad (23.1)$$

where w_i is the priority weight of constraint i and $v_i(\mathbf{x})$ is the violation measure (0 if satisfied, positive otherwise).

Neighbourhood Structure A neighbour state $\mathbf{x}'$ is generated by applying a random perturbation δ to the current state:

$$\mathbf{x}' = \mathbf{x} + \delta, \quad \delta \sim \mathcal{U}(-\epsilon, \epsilon) \quad (23.2)$$

where ϵ is the maximum perturbation magnitude, adaptively scaled based on acceptance history.

Acceptance Probability The Metropolis acceptance criterion determines whether to move to a neighbour state:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } f(\mathbf{x}') \leq f(\mathbf{x}) \\ \exp\left(-\frac{f(\mathbf{x}') - f(\mathbf{x})}{T}\right) & \text{if } f(\mathbf{x}') > f(\mathbf{x}) \end{cases} \quad (23.3)$$

where $T > 0$ is the current temperature parameter.

Cooling Schedule The temperature decreases according to a geometric cooling schedule:

$$T_{k+1} = \alpha \cdot T_k, \quad \alpha \in (0, 1) \quad (23.4)$$

with typical values $\alpha \in [0.95, 0.999]$. The initial temperature T_0 is set such that the initial acceptance probability for a worst-case perturbation is approximately 0.8:

$$T_0 = -\frac{\Delta f_{\max}}{\ln(0.8)} \quad (23.5)$$

Convergence Criterion The algorithm terminates when one of the following conditions is met:

- Temperature threshold: $T_k < T_{\min}$ (typically 10^{-6})
- No improvement: $f(\mathbf{x}_{\text{best}})$ has not improved for N_{stale} iterations
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0$ randomly, set $T_0, k = 0$
2. Generate neighbour $\mathbf{x}'$ from $\mathbf{x}_k$
3. Compute $\Delta f = f(\mathbf{x}') - f(\mathbf{x}_k)$
4. If $\Delta f \leq 0$ or $\mathcal{U}(0, 1) < \exp(-\Delta f/T_k)$, set $\mathbf{x}_{k+1} = \mathbf{x}'$
5. Otherwise, set $\mathbf{x}_{k+1} = \mathbf{x}_k$
6. Update $T_{k+1} = \alpha \cdot T_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot |\mathcal{N}(\mathbf{x})|)$ where $|\mathcal{N}(\mathbf{x})|$ is the neighbourhood evaluation cost
- Probability of finding global optimum approaches 1 as $k_{\max} \rightarrow \infty$ with sufficiently slow cooling
- Space complexity: $O(|\mathbf{x}|)$ for storing the current and best states

Genetic Algorithm (ga)

An evolutionary optimization method that maintains a population of candidate solutions and evolves them through biologically-inspired operators: selection, crossover, and mutation.

Population Representation The population at generation t is a set of N chromosomes:

$$P(t) = \{\mathbf{x}_1^{(t)}, \mathbf{x}_2^{(t)}, \dots, \mathbf{x}_N^{(t)}\} \quad (23.6)$$

Each chromosome $\mathbf{x}_i^{(t)} \in \mathcal{X}$ encodes a complete layout state as a gene vector:

$$\mathbf{x}_i^{(t)} = (g_1, g_2, \dots, g_m) \quad (23.7)$$

where each gene g_j represents a design variable (position, route choice, etc.).

Fitness Function The fitness of chromosome $\mathbf{x}_i$ is derived from the objective function:

$$\text{fit}(\mathbf{x}_i) = \frac{1}{1 + f(\mathbf{x}_i)} \quad (23.8)$$

where $f(\mathbf{x}_i)$ is the constraint violation cost. Higher fitness indicates better solutions.

Selection Parent selection uses tournament selection of size τ :

1. Sample τ individuals uniformly from $P(t)$
2. Select the individual with highest fitness as parent

This is repeated to select N parents for mating.

Crossover Single-point crossover with probability p_c :

$$\text{crossover}(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} (g_1^{(i)}, \dots, g_c^{(i)}, g_{c+1}^{(j)}, \dots, g_m^{(j)}) & \text{if } \mathcal{U}(0, 1) < p_c \\ \mathbf{x}_i & \text{otherwise} \end{cases} \quad (23.9)$$

where $c \sim \mathcal{U}\{1, m-1\}$ is the crossover point.

Mutation Per-gene mutation with probability p_m :

$$g'_j = \begin{cases} g_j + \mathcal{N}(0, \sigma^2) & \text{if } \mathcal{U}(0, 1) < p_m \\ g_j & \text{otherwise} \end{cases} \quad (23.10)$$

where σ is the mutation strength, often adapted over generations.

Elitism The best $\lfloor e \cdot N \rfloor$ individuals are copied unchanged to the next generation, where $e \in [0, 0.2]$ is the elitism rate.

Convergence Criterion The algorithm terminates when:

- Maximum generations: $t > t_{\max}$
- Fitness stagnation: Best fitness has not improved for t_{stale} generations
- Target fitness: $\text{fit}(\mathbf{x}_{\text{best}}) > 1 - \epsilon_{\text{fit}}$

Algorithm

1. Initialize population $P(0)$ randomly
2. Evaluate fitness for all $\mathbf{x}_i^{(t)} \in P(t)$
3. Apply elitism: copy top $e \cdot N$ individuals to $P(t+1)$
4. Select parents via tournament selection
5. Apply crossover with probability p_c
6. Apply mutation with probability p_m
7. Replace population: $P(t) \leftarrow P(t+1)$
8. If convergence not met, $t \leftarrow t+1$, goto 2

Properties

- Time complexity: $O(t_{\max} \cdot N \cdot m)$ for N individuals of length m
- Maintains population diversity, reducing local optima entrapment
- Space complexity: $O(N \cdot |\mathbf{x}|)$ for storing the population
- Naturally parallelizable across individuals

Constraint Programming (cp)

A declarative optimization approach that models the problem as a set of variables, domains, and constraints, using propagation and backtracking to find solutions.

Problem Formulation A constraint satisfaction problem (CSP) is defined as a tuple:

$$\mathcal{P} = (\mathcal{X}, \mathcal{D}, \mathcal{C}) \tag{23.11}$$

where:

- $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ is the set of decision variables
- $\mathcal{D} = \{D(x_1), D(x_2), \dots, D(x_n)\}$ is the set of variable domains
- $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$ is the set of constraints

Domains Each variable x_i has an initial domain $D(x_i) \subseteq \mathbb{R}$ (or discrete set). For PCB layout:

- Position variables: $D(x_i) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$
- Route variables: $D(x_i) = \{\text{path}_1, \text{path}_2, \dots\}$

Constraint Types Constraints are classified as:

- **Unary:** $c(x_i)$ — restricts a single variable
- **Binary:** $c(x_i, x_j)$ — relates two variables
- **Global:** $c(x_{i_1}, \dots, x_{i_k})$ — involves multiple variables (e.g., `alldiff`)

Arc Consistency (AC-3) For each binary constraint $c(x_i, x_j)$, enforce arc consistency:

$$\forall v_i \in D(x_i), \exists v_j \in D(x_j) \text{ such that } c(v_i, v_j) = \text{true} \quad (23.12)$$

If no such v_j exists, remove v_i from $D(x_i)$. Repeat until no more domains can be reduced.

Propagation When a domain is reduced, propagate effects to all connected constraints:

1. Maintain a worklist W of affected variables
2. For each $x_i \in W$, re-evaluate all constraints involving x_i
3. If any domain $D(x_j)$ is reduced, add x_j to W
4. Continue until W is empty (fixpoint reached) or a domain becomes empty (failure)

Backtracking Search The solver explores the search tree depth-first:

1. Select unassigned variable x_i (branching heuristic: smallest domain first)
2. For each value $v \in D(x_i)$ (best-first ordering):
 - (a) Assign $x_i \leftarrow v$
 - (b) Propagate constraints
 - (c) If propagation succeeds (no empty domains), recurse
 - (d) If propagation fails, backtrack

Optimization Extension For optimization (minimizing $f(\mathbf{x})$), add a bound constraint:

$$f(\mathbf{x}) < f(\mathbf{x}_{\text{best}}) \quad (23.13)$$

This is tightened each time a better solution is found (branch-and-bound).

Convergence Criterion

- Solution found: All variables assigned, all constraints satisfied
- No solution: Search tree exhausted (problem infeasible)
- Optimality: All nodes pruned, current best is optimal

Properties

- Time complexity: Exponential in worst case, but propagation prunes large portions of the search tree
- Guarantees constraint satisfaction (complete when solution exists)
- Space complexity: $O(n \cdot |D_{\max}| + m)$ for domain storage and constraint graph
- Deterministic: Same problem instance yields same solution

Gradient Descent (gd)

A first-order iterative optimization algorithm that finds local minima by moving in the direction of steepest descent of the objective function.

Objective Function Assume the objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable, where n is the number of design variables (component positions, trace parameters, etc.).

Gradient Computation The gradient of f at point $\mathbf{x}_k$ is:

$$\nabla f(\mathbf{x}_k) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^\top \quad (23.14)$$

Computed via automatic differentiation or finite differences:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}_k + h\mathbf{e}_i) - f(\mathbf{x}_k - h\mathbf{e}_i)}{2h} \quad (23.15)$$

where h is a small perturbation and $\mathbf{e}_i$ is the i -th unit vector.

Update Rule The basic gradient descent update is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta_k \nabla f(\mathbf{x}_k) \quad (23.16)$$

where $\eta_k > 0$ is the learning rate at iteration k .

Learning Rate Schedules

- **Constant:** $\eta_k = \eta$
- **Decay:** $\eta_k = \eta_0 / (1 + \beta k)$
- **Exponential:** $\eta_k = \eta_0 \cdot \gamma^k$
- **Adaptive:** $\eta_{k,i} = \eta_0 / \sqrt{s_i + \epsilon}$ (per-parameter, AdaGrad-style)

where η_0 is the initial learning rate, β, γ are decay parameters, s_i accumulates past squared gradients, and ϵ prevents division by zero.

Momentum Enhancement To accelerate convergence and dampen oscillations:

$$\mathbf{v}_{k+1} = \mu \mathbf{v}_k + \eta_k \nabla f(\mathbf{x}_k) \quad (23.17)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{v}_{k+1} \quad (23.18)$$

where $\mu \in [0, 1]$ is the momentum coefficient (typical: 0.9).

Constraint Handling For constraints $g_j(\mathbf{x}) \leq 0$, use a penalty method:

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}) + \lambda \sum_{j=1}^m \max(0, g_j(\mathbf{x}))^2 \quad (23.19)$$

where λ is a penalty weight, increased iteratively:

$$\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max}), \quad \rho > 1 \quad (23.20)$$

Convergence Criterion

- Gradient norm: $\|\nabla f(\mathbf{x}_k)\| < \epsilon_{\text{grad}}$
- Parameter change: $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| < \epsilon_{\text{step}}$
- Function change: $|f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k)| < \epsilon_f$
- Maximum iterations: $k > k_{\max}$

Algorithm

1. Initialize $\mathbf{x}_0, \eta_0, \lambda_0, k = 0$
2. Compute $\nabla f(\mathbf{x}_k)$ and $\nabla g_j(\mathbf{x}_k)$
3. Update penalty: $\lambda_{k+1} = \min(\lambda_k \cdot \rho, \lambda_{\max})$
4. Compute search direction: $\mathbf{d}_k = -\nabla \tilde{f}(\mathbf{x}_k)$
5. Line search: find η_k satisfying Armijo condition
6. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \eta_k \mathbf{d}_k$
7. If convergence not met, $k \leftarrow k + 1$, goto 2

Properties

- Time complexity: $O(k_{\max} \cdot n)$ per iteration for gradient computation
- Converges to local minima; requires convexity or good initialization for global optimality
- Space complexity: $O(n)$ for storing the current point and gradient
- Fast per-iteration, but may require many iterations for convergence

Hybrid (hybrid)

A multi-strategy approach that combines the strengths of different algorithms in a sequential pipeline, adapting the optimization strategy to the problem characteristics at each phase.

Architecture The hybrid solver operates in three sequential phases:

$$\mathbf{x}_{\text{final}} = \text{Phase}_3(\text{Phase}_2(\text{Phase}_1(\mathbf{x}_0))) \quad (23.21)$$

Phase 1: Global Exploration (Simulated Annealing)

- Algorithm: Simulated annealing with reduced iterations ($k_{\text{max}}^{(1)} = k_{\text{max}}/4$)
- Purpose: Explore the solution space broadly, identify promising regions
- Output: $\mathbf{x}_1$ with $f(\mathbf{x}_1) < f_{\text{threshold}}$

The temperature schedule uses faster cooling ($\alpha = 0.95$) to quickly identify basins of attraction.

Phase 2: Local Refinement (Gradient Descent)

- Algorithm: Gradient descent with momentum
- Purpose: Refine the solution from Phase 1 to a local minimum
- Input: $\mathbf{x}_1$ from Phase 1
- Output: $\mathbf{x}_2$ with $\|\nabla f(\mathbf{x}_2)\| < \epsilon_{\text{grad}}$

The penalty weight is initialized from Phase 1's final value and continues to increase.

Phase 3: Constraint Satisfaction (Constraint Programming)

- Algorithm: Constraint propagation and backtracking
- Purpose: Verify and enforce all hard constraints
- Input: $\mathbf{x}_2$ from Phase 2
- Output: $\mathbf{x}_{\text{final}}$ feasible or infeasible report

If $\mathbf{x}_2$ violates hard constraints, CP attempts repair. If repair fails, the solver reports the conflict set.

Adaptive Phase Selection Based on problem characteristics, the hybrid solver may skip phases:

$$\text{skip}(\text{Phase}_i) = \begin{cases} \text{true} & \text{if } n < n_{\text{simple}} \text{ and } i = 1 \\ \text{true} & \text{if all constraints linear and } i = 1 \\ \text{false} & \text{otherwise} \end{cases} \quad (23.22)$$

Phase Transition Criteria Transition from Phase i to Phase $i + 1$ occurs when:

- Improvement rate falls below threshold: $\Delta f / \Delta k < \epsilon_{\text{rate}}$
- Maximum phase iterations reached: $k^{(i)} > k_{\text{max}}^{(i)}$
- Solution quality threshold met: $f(\mathbf{x}) < f_{\text{threshold}}^{(i)}$

Algorithm

1. Initialize $\mathbf{x}_0$, determine phase parameters from problem size n
2. **Phase 1:** Run SA with $k_{\text{max}}^{(1)}$, obtain $\mathbf{x}_1$
3. **Phase 2:** Run GD from $\mathbf{x}_1$ until convergence, obtain $\mathbf{x}_2$
4. **Phase 3:** Run CP from $\mathbf{x}_2$, verify/repair constraints
5. Return $\mathbf{x}_{\text{final}}$ or infeasibility report

Properties

- Time complexity: Sum of phase complexities, typically $O(k_{\text{max}}^{(1)} \cdot |\mathcal{N}| + k_{\text{max}}^{(2)} \cdot n + \text{CP}_{\text{search}})$
- Combines global exploration with local exploitation and constraint guarantees
- Adapts to problem complexity via phase skipping
- Space complexity: Maximum of individual phase requirements

23.2.2 Default Algorithm

If no algorithm is specified, the solver uses simulated annealing (**sa**) as the default.

23.3 Constraint Resolution

The solver resolves physical constraints according to the following rules:

23.3.1 No Conflicts

If all constraints can be satisfied without conflicts, the solution is complete.

23.3.2 Conflicts Between Different Priorities

When constraints with different priorities conflict:

- The higher-priority constraint is satisfied
- The lower-priority constraint is optimized as much as possible without impairing the higher rule
- A warning is printed to the console

23.3.3 Conflicts Between Same Priorities

When constraints with the same priority conflict, behavior depends on the priority level:

Lower Priorities (-d, -nth, -imp, -vimp)

- An error message is printed to the console
- The program continues execution

Higher Priorities (-oeo, -eo, -ne, -nonneg)

- An error message is printed to the console
- The program does NOT continue execution (halts)

23.4 Layout Generation Pipeline

The final layout is generated through a multi-phase pipeline:

23.4.1 Phase 1: Constraint Collection

All physical constraints are collected from the program and organized by type and priority.

23.4.2 Phase 2: Board Setup

The board stackup is defined (layers, thicknesses, materials) and the design area is initialized.

23.4.3 Phase 3: Component Placement

Parts are placed on the board according to:

1. Alignment constraints (`phys.align`)
2. Proximity constraints (`phys.prox`)
3. Layer constraints (`phys.layer`)
4. Creepage constraints (`phys.creep`)

The solver optimizes placement to satisfy all constraints simultaneously, respecting the priority hierarchy.

23.4.4 Phase 4: Trace Routing

Connections are routed as traces on the board:

1. Current capacity constraints (`phys.connectioncurrent`) determine minimum trace width
2. Connection matching constraints (`phys.connmatch`) ensure length/impedance matching
3. Net shrinkage (`phys.shrinknet`) minimizes trace length

23.4.5 Phase 5: Via Placement

Vias are placed to connect traces across layers:

1. Via fill constraints (`phys.viafill`) populate thermal vias
2. System flags (`syst.flag.vias`, etc.) control via availability

23.4.6 Phase 6: Plane Generation

Copper planes are generated on specified layers:

1. Plane constraints (`phys.mkplane`) create ground and power planes
2. Clearances are maintained around non-connected objects

23.4.7 Phase 7: Design Rule Check

The final layout is verified against all constraints:

1. All physical constraints are checked
2. Violations are reported with severity based on priority
3. Critical violations (`-ne`, `-nonneg`) halt generation

23.4.8 Phase 8: File Export

The verified layout is exported to KiCad format:

1. `mkschematic` generates the schematic file (`.kicad_sch`)
2. `mkpcb` generates the PCB file (`.kicad_pcb`)
3. `halt` triggers final export and log file generation

23.5 Ordering of Global Set Constant `*e`

Inside the `*e` pseudo-constant, element order is determined by:

1. Type ordering: part, pin, connection, net
2. Within the same type: order of creation
3. Within pins: pad number order from the footprint

Example:

```
part U1: opamp(type="LM741"); des="";
    fp="Package_DIP:DIP-8_W7.62mm"
net V-
connect V- U1[v-]

# *e = { U1, U1[v+], U1[v-], U1[-], U1[+],
#       U1[out], (U1[v-] | V-), V- }
```

Chapter 24

Flags

System flags control PCB manufacturing capabilities using the `sysflag.` prefix:

`sysflag.<flag_name> <value>`

24.1 Available Flags

- `vias` – Controls whether vias are allowed
- `slots` – Controls whether slots / PCB cuts are allowed
- `jumper` – Controls whether jumper wires are allowed
- `solder_reinforced_tracks` – Controls whether solder-reinforced tracks are allowed
- `buried_vias` – Controls whether buried vias are allowed

Part V

Toolchain

Chapter 25

Compiler and Execution Pipeline

The EDADSL toolchain compiles source code into PCB and schematic files.

25.1 Compilation Phases

1. Lexical Analysis – Tokenization of source code
2. Parsing – Syntax analysis and AST construction
3. Semantic Analysis – Type checking and validation
4. Execution – Program execution with electrical system construction
5. Constraint Solving – Physical layout optimization
6. Code Generation – Output file generation

25.2 Entry Point

Every program must begin with the **start** keyword. The program halts when **halt** is reached or an unrecoverable error occurs.

Chapter 26

Code Generation

26.1 PCB Generation

The `mkpcb` command generates a PCB file based on the circuit and physical constraints:

```
mkpcb
```

This command must be placed after all circuit definitions and physical constraints.

26.2 Schematic Generation

The `mkschematic` command generates a schematic file:

```
mkschematic
```

26.3 Export

The `halt` command terminates execution and exports all created files:

```
halt
```

Chapter 27

Standard Library

Chapter 28

External Library

Chapter 29

Debugging and Logging

29.1 Console Output

The `echo` and `f.print()` functions output to the console:

```
echo $variable  
f.print("Message: ", $variable)
```

29.2 Log File

The `writetolog` function appends messages to the log file:

```
writetolog "Checkpoint reached"
```

The log file is exported when `halt` is executed.

Part VI

Reference

Chapter 30

Exceptions

30.1 Type Errors

Raised when an operation is applied to a value of the wrong type.

30.2 Value Errors

Raised when a function receives an argument outside its domain (e.g., division by zero, invalid list index).

30.3 Reference Errors

Raised when referencing an undefined variable, part, or function.

30.4 Constraint Errors

Raised when physical constraints cannot be resolved (see Section 23).

Chapter 31

Error and Warning Model

31.1 Errors

Errors halt program execution. The error message includes the error type and location.

31.2 Warnings

Warnings are printed for constraint conflicts between different priorities. The program continues execution after warnings.

Chapter 32

Future Improvements

Chapter 33

Implementations

33.1 Reference Implementation

Chapter 34

References

34.1 Language Specification

This document serves as the authoritative specification for the EDADSL language.

34.2 Related Standards

Appendix A

Standard Libraries

A.1 Material Libraries

A.1.1 PCB Substrate Material Library

Common substrate materials:

- FR4 – Standard glass-reinforced epoxy laminate
- Rogers – High-frequency laminates
- Metal-core – Aluminum or copper core substrates

A.1.2 PCB Conductor Material Library

- Copper – Standard conductor (specified in oz/ft²)

A.1.3 PCB Surface-finish Material Library

- HASL – Hot Air Solder Leveling
- ENIG – Electroless Nickel Immersion Gold
- OSP – Organic Solderability Preservative

A.2 Basic Model Libraries

A.2.1 Passive Component Model Library

- R – Resistor (parameters: R, P, type)
- C – Capacitor
- L – Inductor

A.2.2 Active Component Model Library

- MOSFET – MOSFET transistor
- Diode – Standard diode
- BJT – Bipolar junction transistor

A.2.3 Integrated Circuit Component Model Library

- opamp – Operational amplifier
- Additional models in external libraries

A.2.4 Board Hardware Component Model Library

- Connectors
- Mounting holes

A.3 Additional Language Libraries

A.4 Footprint Libraries

Footprints follow KiCad library naming conventions:

`<Library>:<Footprint>`

Examples:

`Resistor_THT:R_Axial_DIN0309_L9.0mm_D3.2mm_P12.70mm_Horizontal`

`Package_T0_SOT_THT:T0-247-3_Horizontal_TabUp`

`Package_DIP:DIP-8_W7.62mm`

Appendix B

Examples

B.1 LED Circuit

B.1.1 Intent

A simple LED current-limiting circuit.

B.1.2 EDADSL Code

```
start

part "R1": R(R=330, P=0.25);
    des="330 ohm resistor";
    fp="Resistor_THT:R_Axial_DIN0309"
part "D1": LED(color="red");
    des="Red LED";
    fp="LED_THT:LED_D3.0mm"

net "Vcc"
net "Gnd"

connect Vcc R1[1]
connect R1[2] D1[a]
connect D1[c] Gnd

halt
```

B.1.3 Explanation

The circuit connects a 330Ω resistor in series with an LED between Vcc and Gnd. The resistor limits current to approximately 10mA (assuming 5V supply and 2V LED forward voltage).

Appendix C

Diagrams

C.1 Execution-phase Diagramm